Universidad Autónoma de Baja California
FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA

Inteligencia Artificial

Práctica No. 3: Implementación de Breadth-First Search (BFS).

Alumno:
Manuel A. Cruz L. – 1298063
Joshua Osorio O. – 1293271
Docente:
Mauricio Alonzo Sanchez Herrera.
27/03/2026

Metodología a utilizar: Breadth-first search....7
Introducción....8
Objetivo general....8
Marco teórico....8
Metodología....8
Algoritmo BFS....8
Tecnologías....8
Implementación....8
Resultados esperados....8
Conclusión....8
Bibliografía....8

Introducción

En el área de la inteligencia artificial, los algoritmos de búsqueda representan una herramienta fundamental para resolver problemas relacionados con rutas, toma de decisiones y exploración de espacios de estados. Uno de los problemas más comunes consiste en encontrar el camino más corto entre un punto de origen y un destino dentro de un conjunto de poblaciones conectadas por carreteras.

Para este caso, se implementará el algoritmo Breadth-First Search (BFS), también conocido como búsqueda en anchura, con el objetivo de encontrar la ruta más corta entre dos localidades dentro de un espacio de búsqueda definido previamente.

El algoritmo BFS pertenece a las técnicas de búsqueda no informada, ya que no utiliza información adicional sobre la cercanía al objetivo, sino que explora sistemáticamente todos los nodos vecinos nivel por nivel hasta encontrar el destino.

Objetivo

Analizar un sistema de búsqueda de rutas mediante la implementación del algoritmo Breadth-First Search (BFS), con la finalidad de encontrar el camino más corto entre un punto inicial y un destino, aplicando conceptos de inteligencia artificial y estructuras de grafos

Marco teórico

Inteligencia artificial y búsqueda en grafos

La inteligencia artificial utiliza algoritmos de búsqueda para resolver problemas donde el espacio de soluciones puede representarse como un un grafo o árbol. Estos grafos están compuestos por nodos que representan estados y aristas que representan acciones o transiciones.
Los algoritmos de búsqueda permiten explorar estos espacios con el objetivo de encontrar un camino desde un estado inicial hasta un estado objetivo.

Breadth first search

Breadth first search es un algoritmo de búsqueda no informado que explora el espacio de estos por niveles. Primero se expande el nodo raíz, posteriormente sus sucesores, y así sucesivamente.
De Acuerdo con Russell y Norving, cuando todas las acciones tienen el mismo costo, el algoritmo no garantiza encontrar la solución con menor número de pasos, siendo considerado un algoritmo completo y óptimo.

Propiedades del algoritmo

- Completo
- Óptimo (Cuando los costos son iguales)
- Utiliza estructura de datos tipo cola FIFO
- Explora nodos por nivel

Metodología

1. La metodología para el desarrollo del proyecto se divide en las siguientes etapas:
2. Definición del grafo de ciudades
3. Representación mediante listas de adyacencia
4. implementación del algoritmo BFS
5. Desarrollar de visualización con PyGames
6. Integración del algoritmo con la visualización
7. pruebas del sistema
8. Análisis de resultados

Algoritmo BFS

```
C/C++
#include <stdio.h>
#include “graph.h”
#include “queue.h”
#define A 0
#define B 1
#define C 2
#define D 3
#define E 4
#define F 5
#define G 6
#define H 7
int init_graph( graph_t *g_p )
{
addEdge( g_p, A, B, 1 );
addEdge( g_p, A, C, 1 );
addEdge( g_p, B, D, 1 );
addEdge( g_p, C, E, 1 );
addEdge( g_p, C, F, 1 );
addEdge( g_p, D, G, 1 );
addEdge( g_p, D, H, 1 );
return 0;
}
void bfs( graph_t *g_p, int root, int goal )
{
int node;
int to;
queue_t *q_p;
q_p = createQueue( 10 );
enQueue( q_p, root );
while ( !isEmptyQueue(q_p) ) {
node = deQueue( q_p );
printf(“%d\n”, node);
if (node == goal) break;
for (to = g_p->nodes-1 ; to > 0 ; to--) {
if (getEdge( g_p, node, to ) ) {
enQueue( q_p, to );
}
}
}
destroyQueue( q_p );
return;
}
int main()
{
```

```
Uninformed Search 41
graph_t *g_p;
g_p = createGraph( 8 );
init_graph( g_p );
bfs( g_p, 0, 7 );
destroyGraph( g_p );
return 0;
}
```

Tecnologías

A pesar de que el código en el libro se presenta en lenguaje C, se optara por utilizar la plataforma de desarrollo Python3 debido a su versatilidad, y la implementación de librerías especializadas en la visualización y cómputo científico además que cuenta con una sintaxis que favorece con la legibilidad del código.

Previamente ya se había trabajado con esta plataforma, por lo cual buscamos replicar un funcionamiento similar al proyecto pasado, utilizando librerías como, pygames, la cual será necesaria para poder visualizar los recorridos y cambios que se hagan en la búsqueda dentro del grafo, también se utiliza la librería collections para poder trabajar de manera más ordenada y fácil, el tema de la cola (FIFO), ya que esta librería ya incluye la modulación de esta herramienta, y por último se utilizará la librería time, para poder saber el tiempo en el que terminó el recorrido, y poder manejar la velocidad en la que están realizando las búsquedas.

Lenguaje: Python
Librerías: PyGames, Collection, Time
Herramientas: Visual Studio Code, Git, Entorno virtual.

Resultados esperados

Lo que se espera para este proyecto es poder encontrar una ruta de ciudad A a ciudad B, haciendo que nuestro algoritmo pase por cada uno de los nodos, para encontrar la solución deseada.

Se espera obtener:

1. Visualización del grafo
2. Animación del algoritmo
3. Expansión por nodos por nivel
4. Identificación del camino encontrado

Conclusión

La implementación del algoritmo Breadth first search permitirá analizar el comportamiento de los algoritmos de búsqueda no informada y su aplicación en problemas reales de navegación. La visualización mediante PyGames facilitará la comprensión del proceso de búsqueda y permitirá observar el recorrido del algoritmo de manera gráfica y dinámica.

Bibliografía

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Higher Ed, 2021.
2. M. T. Jones, *Artificial intelligence: A Systems Approach*. Infinity Science PressLlc, 2008.